



Objective & Lecture Mapping: Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

Part 1: Code Analysis & PEAS Evaluation

Task 0 : Setting up and getting the starter Code

1. Go to the [Git Hub Repo](#). Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" brach of the repo.

Task 1.1: The Environment State (`_init_`)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

P – Performance Measure,E – Environment,A – Actuators,
S – Sensors

2. (Understand) Look at the `VisualGridHuntGame._init_` method. Which specific variables in this code represent the physical "Environment (E)" state?

```
self.width,self.height,self.agent_pos,self.opponents
self.walls
self.food_positions
```

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

The baseline environment is Multi-Agent.The environment includes the main agent and a list of opponent agents stored in self.opponents. These opponents are created based on the num_opponents parameter and move independently during the game in the execute_action() method. Because multiple agents (the player and the opponents) interact within the same environment, it is classified as a Multi-Agent environment.

Task 1.2: The Perception Subsystem (`get_percept`)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

Percept Sequence: The absolute, cumulative historical log of every single percept the agent has received from its initialization up to the current moment.

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

The `get_percept()` method represents the Sensors (S) in the PEAS framework because it collects information from the environment and provides the agent with its current percepts.

6. (Evaluate) Based on the exact dictionary returned by `get_percept()` , is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.

This environment is Partially Observable because the agent does not receive complete information about the environment. The `get_percept()` method provides the agent's position, opponent positions, collision status, score, and remaining food, but it does not reveal the locations of all food items or walls. Therefore, the agent cannot fully observe the entire environment at any given time.

Task 1.3: Action Execution (`execute_action`)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

The Performance Measure (P) is being updated because hitting a wall decreases the agent's score, which measures how well the agent is performing.

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

It is important because the Performance Measure should evaluate the agent based on its actual results in the environment, not its internal computation. Hitting a wall is an external outcome that affects the agent's success, making the evaluation objective and focused on achieving the task.

Part 2: Guided Modification & Deep Theory Mapping

Follow the Implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

Step 2.1: Extending the Environment Initialization (`_init_`)

1. **Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `_init_` . Populate it with coordinate tuples safely avoiding position `(0, 0)` , walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally **hide** this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?

Hiding `self.toxic_traps` from the agent's sensors makes the environment Partially Observable because the agent cannot directly perceive the locations of the traps and must act without complete information about the environment.

Step 2.2: Updating the Perception Subsystem (`get_percept`)

1. **Implementation Instructions:** Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin'` : `tuple(self.agent_pos) in self.toxic_traps`) to the dictionary returned to the agent loop.

10. (Analyze) By adding `'smells_toxin'` , you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

Adding 'smells_toxin' improves the agent's rationality because it provides extra information for making better decisions. The agent can avoid toxic traps, reducing negative outcomes and increasing its expected utility (higher score and survival).

Step 2.3: Modifying Action Execution & Visual Rendering (`execute_action` & `draw_grid`)

1. **Implementation Instructions:** In `execute_action` , check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid` ,iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

The classic Vacuum World exploit was when an agent could achieve a high performance score by doing nothing (or avoiding cleaning actions) instead of actually cleaning the environment if the performance metric was poorly designed. It showed that faulty metrics can encourage agents to exploit the scoring system rather than complete the intended task.

Finalize and Lock Lab 01 Analytical Evaluation

Lab 01 code analysis and theoretical evaluation complete and verified!



FACULTY OF COMPUTING